

Unit 5: Hash Functions

Outline

1. Features & Properties
2. Birthday Paradox Attack
3. MD5, SHA-1 and SHA-2 family
4. Introduction to SHA-3

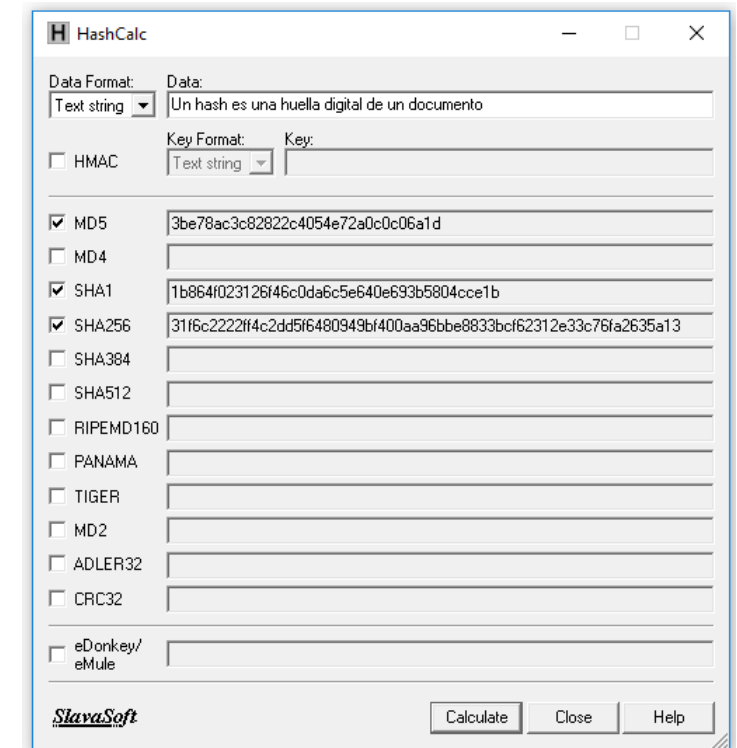
Features & Properties

So far we've seen...

- Key Establishment Protocols
 - Diffie-Hellman and DH on elliptic curves
 - They allow us to agree on a key without the need for a secure channel
- Asymmetric encryption
 - It allows messages to be sent to the recipient without having to negotiate a key beforehand
 - We can also use it to agree on a symmetric encryption key with another entity
- Both allow us to keep information confidential
- What two other security properties do we need to protect?
 - **Integrity**
 - Availability

What is a hash function?

- A hash function (or digest) can be defined as "a function that associates any text or electronic document M of any size, with a fixed-length, supposedly unique $h(M)$ summary."
- The size of $h(M)$ will depend on the algorithm used
- For instance, a hash in MD5 is 128 bits long, 160 bits in SHA-1, and 256 bits in SHA-2



Security in a Hash Functions

- Let's assume a hash function that results in only 4 bits
 - This means that the 16 possible outcomes can be:

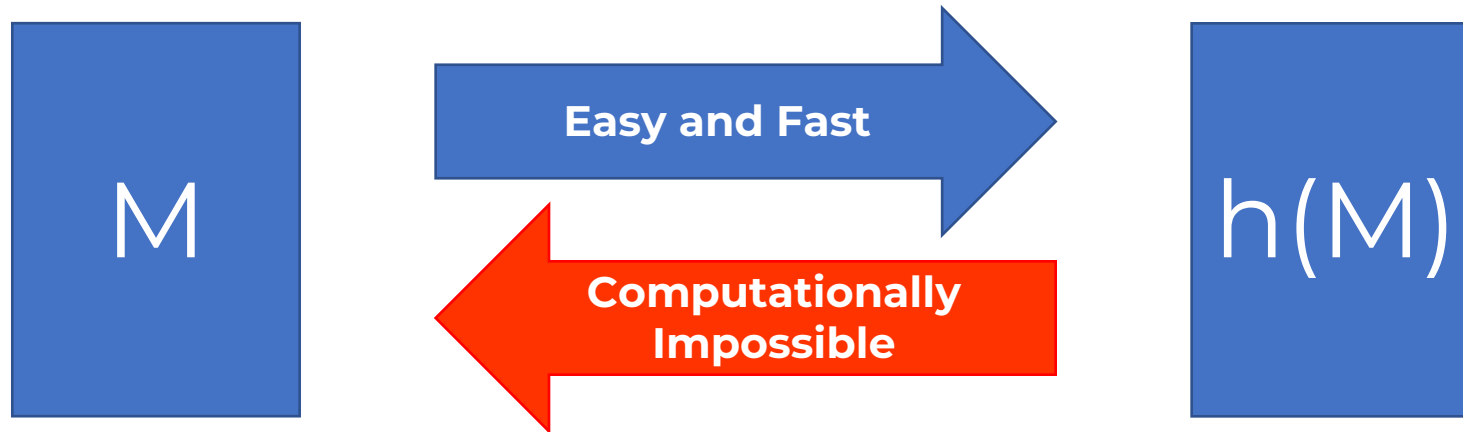
0000	0001	0010	0011	0100	0101	0110	0111
1000	1001	1010	1011	1100	1101	1110	1111

- What is the probability that two different messages have the same hash function?
 - In this example, that probability is $1/16$ because all 16 values are equiprobable.
- We could have these two totally different messages or files with the same hash:
 - “We reject this contract” 22 characters with 1101 hash... and
 - “We accept this contract” also 22 characters long and with the same hash 1101

Properties of hash functions (I)

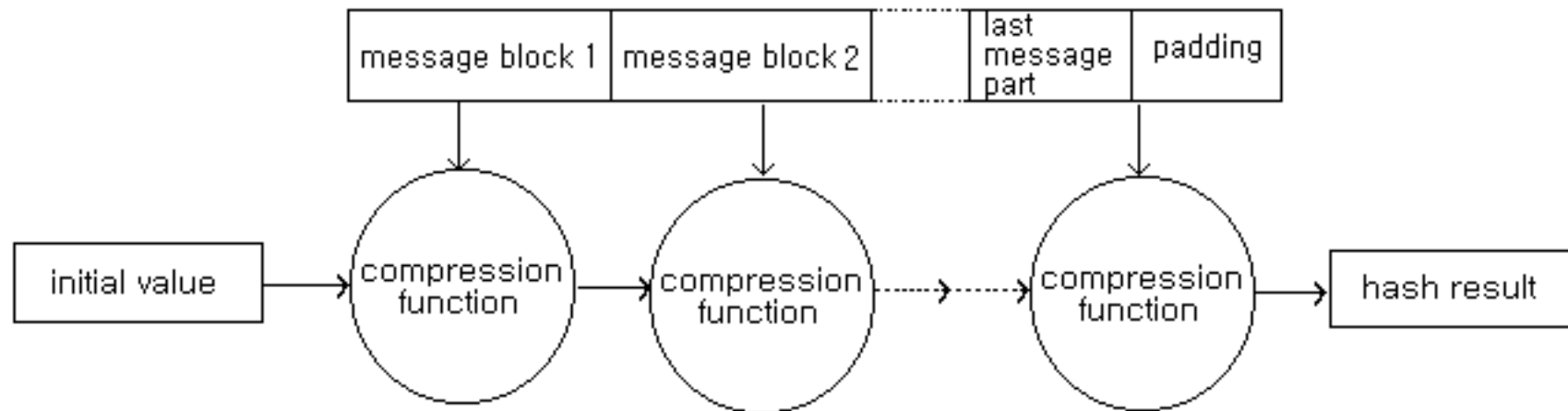
An $h(M)$ hash will be secure if it has the following characteristics:

1. **Ease of calculation:** it should be easy to calculate $h(M)$ from M .
2. **One-way property:** If an $H(M)$ hash is known, it must be computationally impossible to find the message M from the $H(M)$: even if there is a way to solve the problem, the time and resources required should make it unfeasible.



Properties of hash functions (II)

3. **Compression Property:** from a message M of any length, the hash $h(M)$ must have a fixed length. Normally, the length of $h(M)$ is less than the message M .



Properties of hash functions (III)

4. **Diffusion Property:** $h(M)$ must be a function of all bits of the message M . Therefore, if a single bit of the M message is modified, the $h(M)$ hash should change by about half of its bits.

Hash MD5 of M_1 y M_2 which differ in 1 bit (ASCII: **B** = 01000010 and **C** = 01000011)

M_1 = "DRINK **C**OCA COLA" $h(M_1)$ = EBC97CD6472A3FDBE53B09F6E2BA8687

M_2 = "DRINK **B**OCA COLA" $h(M_2)$ = 967A99F55CF64973C553374C08A7BD72

$h(M_1)$ 11101011 11001001 01111100 11010110 01000111 00101010
 00111111 11011011 11100101 00111011 00001001 11110110
 11100010 10111010 10000110 10000111

$h(M_2)$ 10010110 01111010 10011001 11110101 01011100 11110110
 01001001 01110011 11000101 01010011 00110111 01001100
 00001000 10100111 10111101 01110010

70 bits in
red mark
change
between
 $h(M_1)$ and
 $h(M_2)$

Properties of hash functions (IV)

5. Preimage Resistance:

It will be computationally difficult knowing M , to find an M' such that **$h(M) = h(M')$** .

If a hash has n bits, the average probability of this happening is equal to $1/2^{n-1}$. That is, we should average 2^{n-1} attempts, a value very close to 2^n if n is its hundred-bit. All hash functions should comply with this property.

6. Collision Resistance: (Second preimage)

It will be computationally difficult to find a random pair **(M, M') such as $h(M) = h(M')$** .

If a hash has n bits, the average probability of this happening is equal to $1/2^{n/2}$. That is, we should average $2^{n/2}$ attempts, a big reduction due to the birthday paradox. Hash functions are vulnerable to this type of attack

Examples of hash functions

- Parity Bit
- XOR
- Sum of digits mod n

Birthday Paradox Attack

It's easier than it seems to create different messages

- Example of a simple message with $2^{10} = 1.024$ permutations, using 10 synonyms

Dear {Beloved} friend {partner}:
I send you {write you} this letter {note} to tell you {let you know} that your number {lottery ticket} has been awarded {obtained} the main prize {largest prize} of the lottery.
Best {I send you my} regards and wish you a nice day {week}.
Jorge

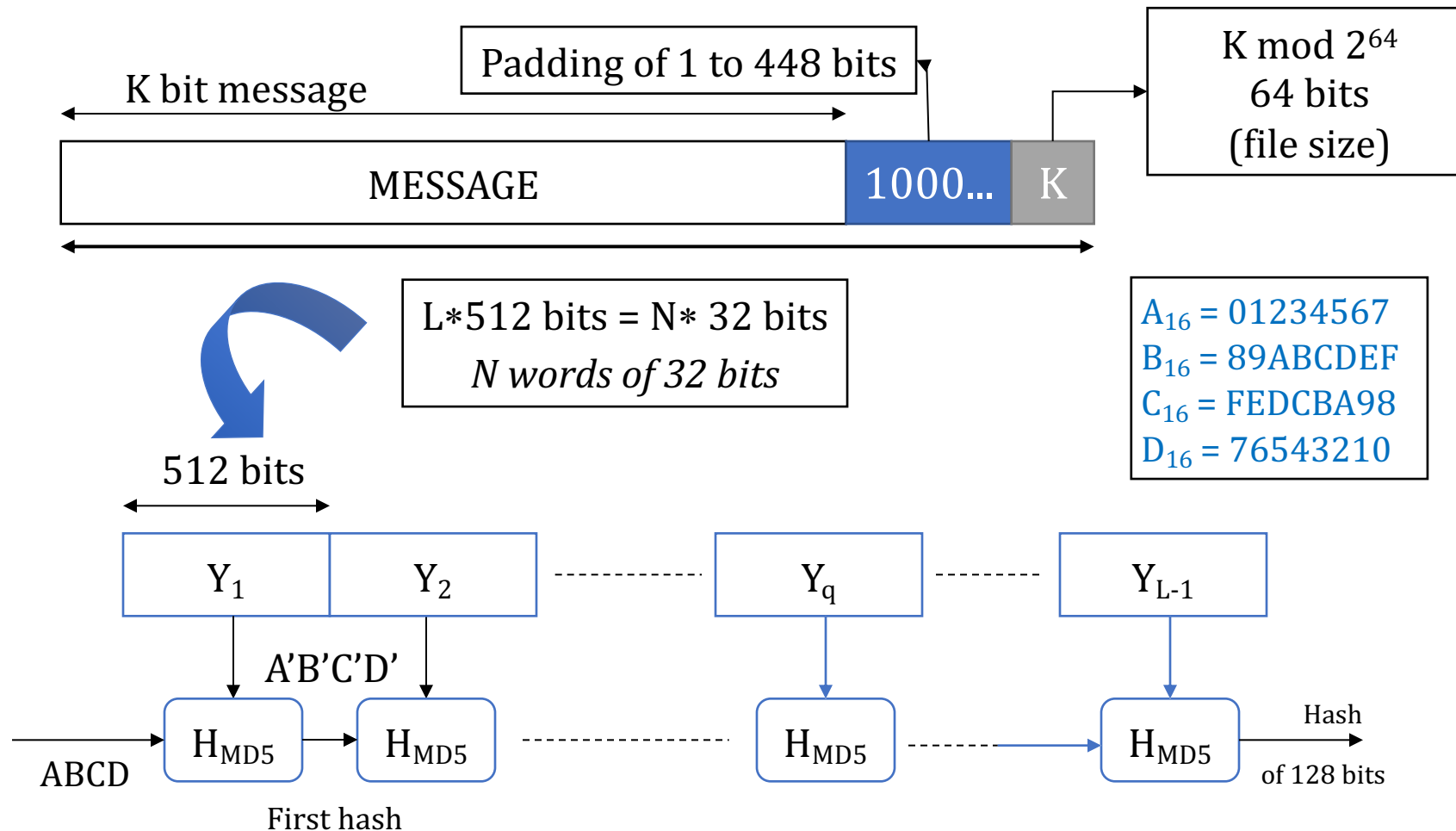
- To find a collision
 1. We create a large number of different messages,
 - Real: That they mean the same thing (that it has been awarded)
 - Fake: That say the opposite (you haven't been awarded)
 2. We look for a collision between their hashes
- If the hash in question has n bits, because of the birthday paradox we only need to average $2^{n/2}$ calculations so that we have a greater than 50% probability of that collision occurring

MD5, SHA-1 and SHA-2

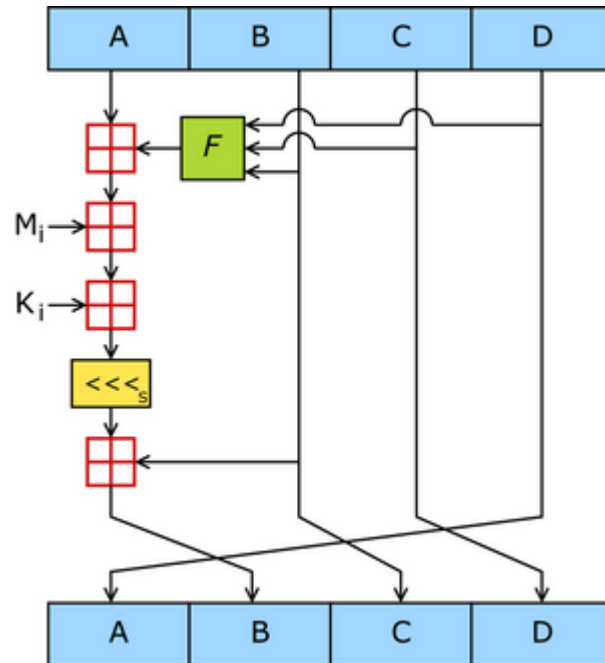
MD5

- Created by Ron Rivest in 1991, this hash has been obsolete since mid-2005. However, it is still used in different applications.
- It processes messages in 512-bit blocks and produces 128-bit output.
- First, it expands the M message or document to a length exactly 64 bits less than a multiple of 512 bits.
- In the last block, we add one bit 1, followed by as many 0s as needed, reserving the last 64 bits to indicate the length of the message in bits.
- We begin with four initial vectors of 32 bits each (A, B, C, and D), whose initial value is public. These vectors and the first block of 512 bits are subjected to 64 nonlinear 32-bit operations.
- At the end, four new vectors A'B'C'D' are obtained, which will be the input vectors for the next block of 512 bits. This action is repeated with the remaining blocks of the document.
- The last ABCD block will give the final 128-bit $h(M)$ summary

Blocks in the MD5 hash function



MD5 Hash Function Scheme



Initial values:

A = 01234567

B = 89ABCDEF

C = FEDCBA98

D = 76543210

Depending on the round, F can be F, G, H, I:

F = (**B** AND **C**) OR (NOT **B** AND **D**) 1st round

G = (**B** AND **D**) OR (**C** AND NOT **D**) 2nd round

H = (**B** XOR **C** XOR **D**) 3rd round

I = (**C** XOR (**B** OR NOT **D**)) 4th round

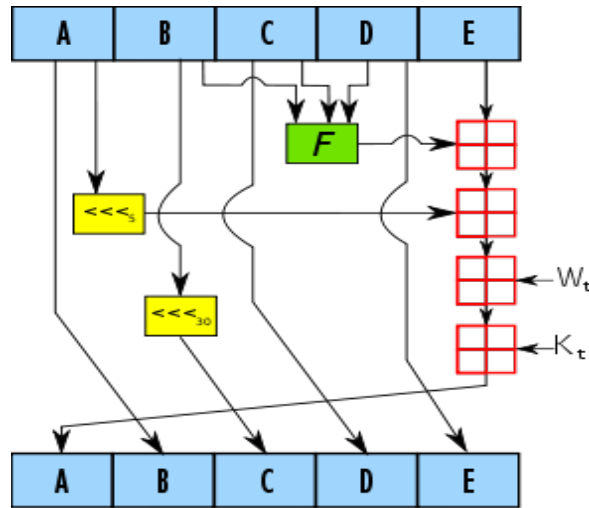
 is a sum mod 2^{32} not an xor

- M_i are 16 32-bit words from the Y_j th block of 512 bits that is used as input for that round
- For each round we do 16 operations to cover the whole block
- K_i is a 32-bit constant (different for each operation that must be looked up in tables)
- s is a bit shift (also different for each round and defined in tables)
- Each Word is 32 bits and is read in Little-endian

The SHA-1 and SHA-2 hash functions

- SHA-1 (Secure Hash Algorithm) created by the NSA in 1995, replaces SHA-0 (1993) in which "certain" problems had been detected.
- SHA-1 is very similar to MD5. It also treats 512-bit blocks of message but generates a 160-bit digest.
- In this case, for each block, a total of 80 operations are performed with 32-bit words, organized into 4 rounds: words 0-19 in the first round... up to words 61-79 in the fourth round.
- Those 80 words are created from the 512-bit block. The first 16 are the block. The next 64 words are obtained with xor operations between the previous words.
- Since it generates 160 bits, it uses 5 non-secret 32-bit ABCDE vectors.
- In 2001, the NSA also developed the SHA-2 family, with 224-, 256-, 384-bit, and 512-bit summaries.
- SHA-256 is today (2024) the most used algorithm in the internet, email clients, applications, smart cards, etc.

SHA-1 hash function schema



Initial values:

A = 67452301

B = EFCDAB89

C = 98BADCFE

D = 10325476

E = C3D2E1F0

Depending on the round F can be F, G, H, I:

F = (B AND C) OR (NOT B AND D)

G = (B XOR C XOR D)

H = (B AND C) OR (B AND D) OR (C AND D)

I = (B XOR C XOR D)

1st round

2nd round

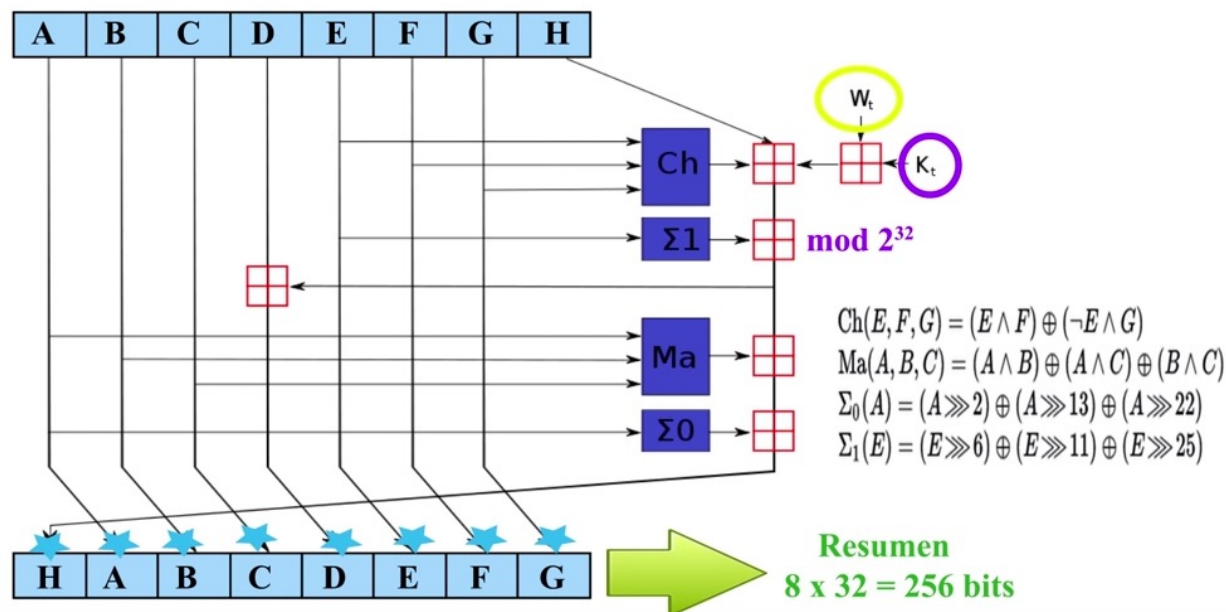
3rd round

4th round

 is a sum mod 2^{32} not an xor

- W_t are 80 32-bit words from block Y_j used as input (previous iteration)
- The first 16 words are from the previous iteration, the next 64 are obtained via xor with previous words
- K_t is a 32-bit constant. There are 4 different constants, one for each round.
- There is a 5 bit left shift over A and a 30 bit left shift over B
- Each word is read in Big-endian

SHA-256 Function Schematic



Algoritmo y variante		Tamaño del hash (bits)	Longitud de la palabra (bits)	Tamaño del bloque (bits)	Vueltas	Tamaño del vector interno (bits)	Tamaño máximo del mensaje (bits)
SHA-2	SHA-224	224	32	512	64	256 (8 x 32)	$2^{64} - 1$
	SHA-256	256					
	SHA-384	384					
	SHA-512	512	64	1.024	80	512 (8 x 64)	$2^{128} - 1$
	SHA-512/224	224					
	SHA-512/256	256					

MD5 and SHA-1 vulnerabilities

- MD5 is 128 bits,
 - There are 2^{128} different hashes
 - A birthday paradox attack would take 2^{64} on average, (very easy today)
- SHA-1 is 160 bits
 - There are 2^{160} different hashes.
 - The birthday paradox attack would need 2^{80} on average, (relatively easy today).
- In 2004, scientists from Shandong University (China) found weaknesses that reduced the collision-finding space in MD5
- Two years later, in 2006, new attacks were published:
 - MD5 lowers strength from 2^{64} to 2^{39} (33 million times)
 - SHA-1 lowered strength from 2^{80} to 2^{63} (130 thousand times)... and keeps going down
 - SHA-1 resisted a little longer, but succumbed on February 23, 2017 with the attack called SHAttered <https://shattered.io/>

Anecdotal case of use of MD5

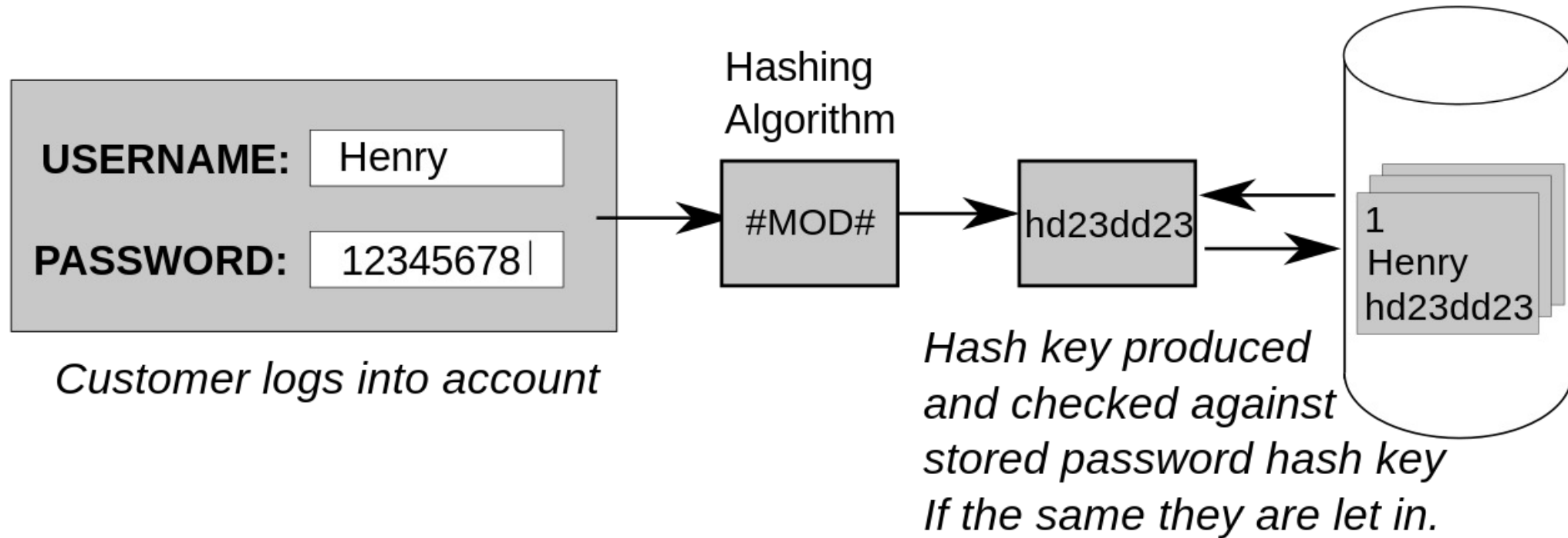
- Australia, August 2005: A radar takes a picture of a speeding car
- The defense attorney appeals the complaint arguing that the image may be of another car (not his client's) or the original may have been altered.
- The police say they use MD5 to check the integrity of the photographs on the device. Since MD5 had problems since 2004, they were not able to find any expert who was willing to prove to the judge the infallibility of this algorithm.
- Conclusion: the complaint was withdrawn.

Use Case: Hash & Passwords

- On systems that require passwords for access, the server **doesn't store the password but its hash.**
- When authenticating, the server calculates the hash of the password sent by the client and compares it with what it has stored
- If someone has access to the hash store, they won't be able to get the password, because of the one-way nature of the hash.
- The "salted hashing" technique adds a few extra bits (the salt) to passwords before the hash is computed.
 - These bits are usually random and are stored together with the hash.
 - This way, it's harder to tell if multiple users are using the same password, or even if the same password is used in more than one place.

Use Case: Hash & Passwords

- Passwords with no salt

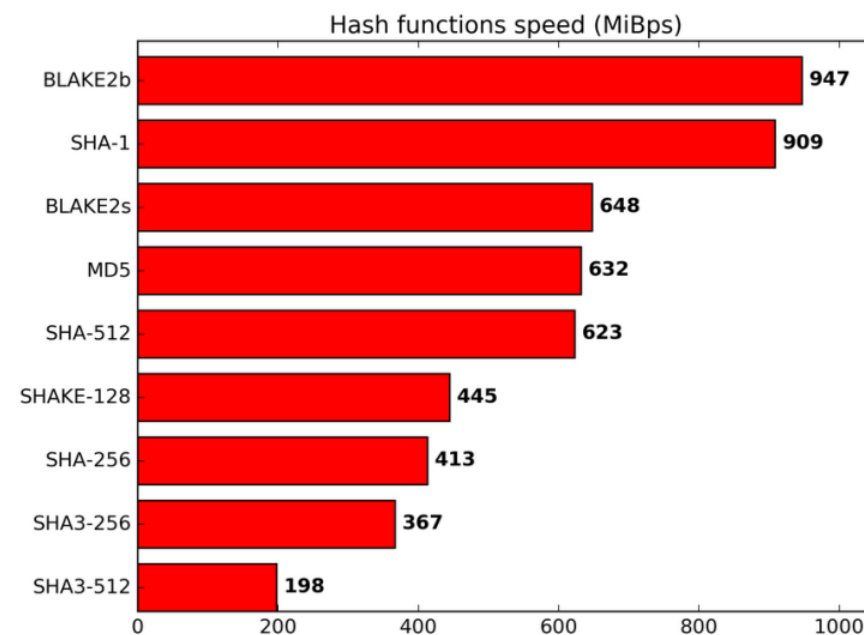


Hash Speed Comparison

```

C:\OpenSSL-Win64\bin>openssl speed md5 sha1 sha256
Doing md5 for 3s on 16 size blocks: 20970984 md5's in 3.02s
Doing md5 for 3s on 64 size blocks: 12851216 md5's in 3.00s
Doing md5 for 3s on 256 size blocks: 5824280 md5's in 3.02s
Doing md5 for 3s on 1024 size blocks: 1840310 md5's in 3.02s
Doing md5 for 3s on 8192 size blocks: 247106 md5's in 3.00s
Doing md5 for 3s on 16384 size blocks: 121765 md5's in 2.98s
Doing sha1 for 3s on 16 size blocks: 21884257 sha1's in 2.98s
Doing sha1 for 3s on 64 size blocks: 13953818 sha1's in 3.00s
Doing sha1 for 3s on 256 size blocks: 7338476 sha1's in 3.02s
Doing sha1 for 3s on 1024 size blocks: 2515177 sha1's in 2.98s
Doing sha1 for 3s on 8192 size blocks: 354860 sha1's in 3.02s
Doing sha1 for 3s on 16384 size blocks: 179098 sha1's in 2.98s
Doing sha256 for 3s on 16 size blocks: 13883009 sha256's in 3.02s
Doing sha256 for 3s on 64 size blocks: 8010755 sha256's in 3.02s
Doing sha256 for 3s on 256 size blocks: 3711694 sha256's in 3.00s
Doing sha256 for 3s on 1024 size blocks: 1204831 sha256's in 3.00s
Doing sha256 for 3s on 8192 size blocks: 160349 sha256's in 2.98s
Doing sha256 for 3s on 16384 size blocks: 79444 sha256's in 2.97s
OpenSSL 1.1.0g  2 Nov 2017
built on: reproducible build, date unspecified
options:bn(64,64) rc4(16x,int) des(long) aes(partial) idea(int) blowfish(ptr)
compiler: cl " " VC-WIN64A
The 'numbers' are in 1000s of bytes per second processed.
type      16 bytes      64 bytes      256 bytes      1024 bytes      8192 bytes      16384 bytes
md5        111265.74k    274159.27k    494430.07k    624904.44k    674764.12k    668480.92k
sha1       117327.12k    297681.45k    622971.97k    863008.59k    963983.63k    983234.89k
sha256     73659.07k     170010.63k    316731.22k    411248.98k    440152.13k    438437.22k

C:\OpenSSL-Win64\bin>
  
```



Done in 2017

Introduction to SHA-3

NIST Proposes New SHA-3 Standard

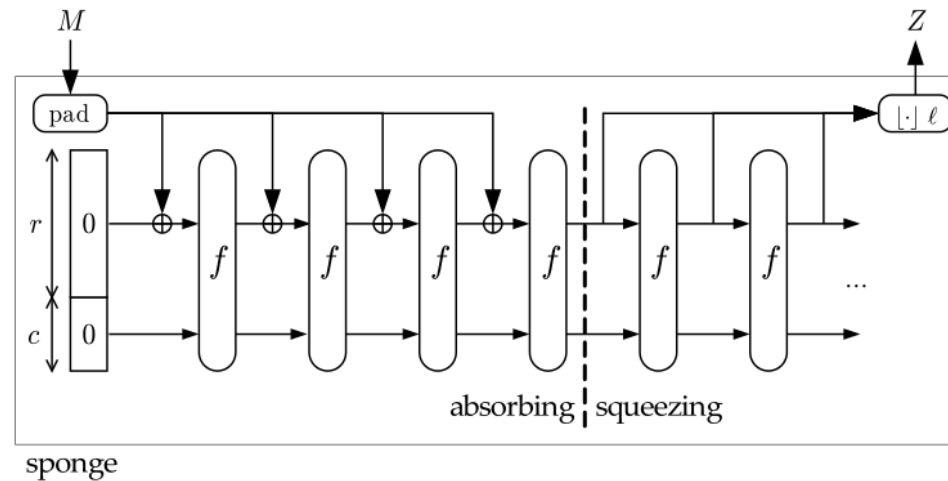
- In view of the attacks to hash functions, in 2005, some cryptographers asked NIST to organise a competition like the one held for AES.
- Finally, on January 23, 2007, NIST issued a press release with a draft for the new hashing standard, which would be called SHA-3.
- Three years later, in December 2010, NIST selected five finalists in the third round: BLAKE, Ghøstl, JH, Keccak and Skein.
- In October 2012, NIST announced that the winner was Keccak, developed by Belgian researchers Joan Daemen, Michael Peeters, Gilles Van Assche, and Italian Guido Bertoni.
- SHA-3 is introduced as a standard by NIST on February 6, 2013.
- In 2024 SHA-3 is a suggestion of use, the SHA-2 family is still safe

<http://csrc.nist.gov/groups/ST/hash/sha-3/index.html>

<http://csrc.nist.gov/groups/ST/hash/sha-3/documents/Keccak-slides-at-NIST.pdf>

Sponge construction(SHA-3)

- Sponge Construction: Function $F(M) = H = h(M)$
 - *Variable Length Input and Variable Length Output*
 - Fixed-length f -permutation (or transformation)
 - Width: number of bits = b
 - *It operates on a state of b bits, where $b = r + c$*
 - r = bits of rate and c = bits of capacity
- M + padding
 - Blocks of r bits
 - S = state (b bits)
 - Initialized to zero
- Two Phases
 - Absorption (input)
 - Squeezing (output)



Summary

- Hash functions take a randomly sized input and produce a constant output
- Properties
 - Ease of calculation
 - Unidirectionality
 - Compression
 - Diffusion
 - Pre image collision resistance
 - Collision resistance
- Birthday Attack
- Relevant Hash functions: MD5, SHA-1, SHA-2

Questions

